



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Advanced Cybersecurity and Artificial Intelligence Topics

LLM-Enhanced Honeypot and Attacker Interaction Analytics

Supervisors: Christos Xenakis, Apostolis Zarras, and Georgios Vouros

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

12/07/2026

Introduction

Honeypots are one of the most instructive defense techniques in cybersecurity. Instead of trying to exclude the attacker, they invite them into a controlled, fake environment that has no real operational value. Because no legitimate user has a reason to touch such a system, any interaction with it is almost by definition suspicious, so it produces clear information about the opponent's tactics and intentions, without the noise of legitimate traffic (lecture 1). The value of a honeypot, however, depends entirely on its persuasiveness. An environment that directly betrays itself does not collect anything, as the attacker leaves as soon as he realizes the trap.

The object of this paper is the construction of an SSH honeypot enriched with content generated by Large Language Models (LLMs), and the systematic analysis of the behavior of attackers interacting with it. The question is not just whether a honeypot can record attacks, but how convincing decoys we can produce, how deeply engaged they cause, and what we can infer about attackers' intentions through unsupervised learning (lecture 3).

A defining design principle of all work is that the analysis should be based on real facts and not on speculation. For this reason, the honeypot we built is not limited to a simple imitation, but simulates a full interactive shell and actually serves the baits. Thus, each metric (for example, which files the attacker opened) results from actual recorded events. This approach is linked to the principles of responsible AI, and in particular explainability and accountability (lecture 7).

The project answers seven questions. The honeypot and the threat model will be designed (Task 1), the decoy artifacts will be produced with an emphasis on LLM-generated (Task 2), structured session recording will be implemented (Task 3), controlled attacker playbooks will be executed (Task 4), clustering and anomaly detection behavior will be analyzed (Task 5), the quality of deception and the risk of fingerprinting will be evaluated (Task 6), and ethical, security and operational use issues will be addressed (Task 7). An optional advanced task supervised intent classification was also implemented.

Structure of the implementation

The system is not a single program, but a chain of distinct scripts in Python, each with a clear responsibility. The **00_config.py** It defines a common system profile (names, versions, addresses, dates) that feeds all the rest, ensuring internal consistency. The

01_generate_decoys.py produces baits. The **02_ssh_honeypot.py** implements honeypot and logging. The **03_run_playbooks.py** executes the attack scenarios. The **04_analyze_sessions.py** It does the behavioral analysis, clustering, and graphing. The **05_intent_classifier.py** implements the optional classifier. A single orchestrator (**run_all.py**) runs the entire chain, both on Linux/Kali and Windows.

The entire stream is repeatable through a fixed random seed (value 25012), so each number in the report results from the same, single execution. The honeypot listens exclusively to the loopback (127.0.0.1) and all data is fictitious.

Task 1: Design and development of the honeypot

The narration does not define a specific scenario but calls for a persuasive environment with a consistent background. The choice of the script was made by us. A standard Linux application server, named app-srv-01, operating Ubuntu 22.04.4 LTS and OpenSSH 8.9p1, was chosen because it is one of the most common real targets. Servers of this type host web applications, connect to databases, and are located on internal corporate networks, so an attacker gaining access to such a server has every reason to explore it.

Unlike a simple banner imitation, the honeypot implements real SSH protocol through the paramiko library, i.e., regular key exchange, authentication, and interactive channel. An attacker connects to a normal ssh client (for example `ssh -p 2222 root@127.0.0.1`) and receives a simulated shell, as would be the case in a real system.

Threat Model

The honeypot is designed to study an attacker who has already gained access to a shell (post-exploitation stage) and explores the system. The objectives we assume are five: identifying the environment, searching for credentials, upgrading privileges, locating data for export, and preparing for horizontal movement.

The term lateral movement describes the attacker's attempt to move from the machine in which he is located to other machines in the same internal network. For example, after gaining access to the application server, the attacker often attempts to connect to the database or other

internal servers to expand their presence. The honeypot is not trying to rule out any of these actions, but to observe and record them.

Architecture

Architecture is organized into four layers, each of which has a distinct role:

1. **The network layer (paramiko Transport):** Accepts the incoming TCP connection, launches the SSH dialog as a server and presents the OpenSSH 8.9p1 banner. At the same time, it records the client banner (client_version), which reveals the tool used by the attacker and is a first fingerprinting element of the adversary itself.
2. **The authentication layer:** The honeypot accepts login attempts (typical honeypot behavior so that the attacker can go ahead and be noticed), but records each attempt, along with the username and method. For security reasons, only a sample of the code is recorded, never in full.
3. **The shell emulator:** This is the most complex layer. It takes each command line, parses it, recognizes the command and its arguments, and executes it on the virtual file system. It supports a realistic subset of commands (ls, cd, cat, grep, find, ps, ss, id, uname, sudo, history, and others), handles paths correctly (absolute, relative, with ~), and keeps state per session, i.e. remembers the current directory so that the successive cd and ls commands behave as in a real shell. Most importantly, when the attacker reads a bait, the simulator actually returns its content and notes that the specific bait was touched.
4. **The Virtual File System (VFS):** It presents the attacker with a seemingly complete file system, combining two sources: decoy artifacts, which are located in real files on disk (decoys/fs/ folder), and synthetic system files (such as /etc/passwd, /etc/os-release, /proc/version) that are dynamically generated by the common profile to be consistent. VFS also includes path traversal protection, preventing the attacker from escaping outside the virtual environment, and dealing with special cases, such as /etc/shadow returning permission denied without root privileges.

Files implementing Task 1: 02_ssh_honeypot.py (honeypot, shell emulator, VFS) and **00_config.py** (system profile). The virtual file system is located in the **decoys/fs/**.

Task 2: Production of decoy artefacts

Twelve (12) decoy artifacts were created in seven (7) categories, exceeding the minimum limits of pronunciation (10 artifacts, 3 categories). Seven (7) of them were produced with LLMs, i.e. with the Claude model, and five (5) manually (template-based), meeting the requirement for at least five LLM-generated artifacts.

The seven baits produced with the Claude model

Each of these seven baits was designed to deliberately reveal a piece of (fictitious) infrastructure, giving the attacker information that motivates him to delve deeper:

- **Deployment account command history (svc_deploy.bash_history):** shows the flow of deployments and lists the internal nodes db01, cache01 and ci01. It acts as a map of how the system works.
- **nginx config (app.corp.conf):** reveals that the application runs locally on port 8000, TLS certificate paths, and an internal endpoint/metrics restricted to the corporate network.
- **Database Configuration File (database.yml):** Reveals the users of the database (app_rw with write permissions, app_ro read-only) and the db01 server. The codes are prominently placeholders, injected during deployment.
- **Authentication Log (auth.log):** shows which users logged in, from which management station, as well as sudo actions and scheduled tasks, enhancing the feel of a living system.
- **.ssh/config:** perhaps the most valuable bait, as it contains a complete map of all internal machines (bastion, db01, cache01, backup01, ci01) and access keys. It guides directly to horizontal movement.
- **Installation Runbook (deploy_runbook.md):** describes the production and rollback process step by step, revealing rich information about the application's architecture and dependencies.
- **Infrastructure Notes (NOTES_infra.md):** written like a manager's hasty human notes. They reveal weaknesses, such as a code that has yet to be changed, creating a sense of urgency and opportunity.

The five template-based baits

The remaining five were created manually and cover more standardized, but equally enticing elements:

- **Administrator's command history (mgeorgiou's `.bash_history`):** shows typical management tasks (service checks, login to the database) and references to other internal systems.
- **Fake credentials file (`.env`):** the most immediately tempting bait, as it seems to contain secrets of the application (base address, keys, codes). However, all values are obviously placeholders (e.g. `CHANGE_ME`, `FAKE`) and point to fictitious nodes.
- **PostgreSQL log:** confirms the version of the database (14) and the name of the database, enhancing the consistency of the story.
- **File of known SSH nodes (`.ssh/known_hosts`):** reveals the internal topology, i.e. which machines exist and with which addresses, guiding this too towards horizontal movement.
- **Scheduled tasks (`cron.d/deploy`):** show the service accounts and scripts that run automatically, elements that an attacker would look for persistence points.

Artefact	Category	Source	Role in history
<code>.bash_history</code> (mgeorgiou)	shell_history	manual	Sysadmin Command History
<code>.bash_history</code> (svc_deploy)	shell_history	Claude	CI/CD deployment stream
<code>app.corp.conf</code>	config	Claude	nginx reverse proxy, /metrics
<code>.env</code>	credentials	manual	Fake secrets (KEY=value)
<code>database.yml</code>	credentials	Claude	Users app_rw/app_ro, db01
<code>postgresql-14-main.log</code>	logs	manual	PostgreSQL Activity
<code>auth.log</code>	logs	Claude	SSH, sudo, cron connections
<code>.ssh/known_hosts</code>	ssh	manual	Internal Network Topology
<code>.ssh/config</code>	ssh	Claude	Fleet map (db01, ci01 etc.)
<code>cron.d/deploy</code>	system	manual	Scheduled tasks
<code>deploy_runbook.md</code>	docs	Claude	Runbook production deployment
<code>NOTES_infra.md</code>	docs	Claude	Admin notes, weaknesses

Method of Production of LLM Artifacts

The content of the seven LLM artifacts was produced with the Claude model, with explicit prompts describing the system profile (names, versions, addresses) and asking for realistic, internally consistent content with no real secrets. Each prompt is stored as it is in the `llm_prompts/` folder for complete documentation and reproducibility. Thus, the content of the LLM baits is stable and integrated, so the execution is always repeatable.

Internal consistency

Internal consistency was a conscious and central design choice, as contradictions are among the most frequent points that betray a honeypot. A cautious attacker compares the information he finds, and a discrepancy (for example, a software version listed differently in two files) is enough to make him suspicious.

To avoid such contradictions, all baits draw their elements from a single source of truth, the common profile (`00_config.py`). So the same host (`app-srv-01`), the same users (`mgeorgiou`, `svc_deploy`), the same versions (OpenSSH 8.9p1, nginx 1.18.0, PostgreSQL 14), and the same internal network (10.20.0.0/16) appear in exactly the same way everywhere. In addition, dates are generated from a fixed reference date rather than the current time, so they don't move in each run and don't contradict texts.

Beyond the consistency of the individual elements, the baits converge into a single, plausible story that the aggressor can reconstruct. For example, the note in `NOTES_infra.md` about the need to switch the password of the user `app_rw` corresponds to the same user `app_rw` that appears in the `database.yml` base configuration file. The installation runbook (`deploy_runbook.md`) describes a process that matches the commands shown in `svc_deploy` account history. The `.ssh/config` file and the `.ssh/known_hosts` file report the same internal nodes (`db01`, `cache01`, `backup01`, `ci01`) with the same addresses. This networked consistency makes the environment look like a real system with a history, rather than a collection of disjointed fake files.

Files implementing Task 2: `01_generate_decoys.py`, the envelope `decoys/` (the artifacts and the `artifact_inventory.json`) and the file `llm_prompts/` (the prompts of the LLM artifacts).

Task 3: Recording and reconstructing sessions

Recording is the foundation of any honeypot: without accurate and structured data, no analysis is possible. The aim of this stage is to record each session in such a way that it can later be completely reconstructed, i.e. to be able to see exactly what the attacker did, in what order, and with what result.

Structure of the recording

Recording is done simultaneously in two forms, for two different uses. Firstly, each session is stored as a standalone, easy-to-read JSON file (logs/sessions/session_<id>.json), suitable for inspecting a single session. Second, all sessions are added to a single master file in JSON Lines format (logs/sessions.jsonl), where each line is a full session. This format is ideal for bulk analysis, as the analyzer reads the file line by line.

For each session, the client's address and port, the start and end timestamps (and therefore the duration), the SSH client banner, all authentication attempts, the full sequence of commands, and the list of decoy artifacts that were touched are recorded. In addition, each command is recorded as a separate event with its serial number, timestamp, current list at the time of execution, response size, and the list of artifacts touched by that command. The full scheme is documented in detail in the SCHEMA.md file.

Touch tracking (artefact tracking)

The most important element of recording is the tracking of touches. Every time a command reads a bait, the honeypot notes it. So a command like `grep -r password /opt/app` can touch two baits (`.env` and `database.yml`) at the same time, and this is accurately captured. Thanks to this, the metric *artefacts touched* becomes a real, measurable quantity, and not a guess. This is what allows, in the next stage, to objectively measure how deeply each attacker was involved with the environment.

In the actual execution, 55 sessions were recorded in total: 35 sessions with commands (the playbooks) and 20 sessions of failed authentication. The latter correspond to the personalities (personas) who first try an incorrect code before logging in. Recording these failed attempts is important as well, as it is a realistic signal of credential probing.

Files implementing Task 3: 02_ssh_honeybot.py (the log), the logs/ (sessions/ and sessions.jsonl) folder and the SCHEMA.md (the full documentation of the schema).

Task 4: Controlled Attacker Playbooks

A playbook is a predetermined attack scenario, i.e. a specific sequence of commands executed by an attacker with a clear objective. For example, a credentials lookup playbook will execute commands that look for codes and keys. Because we didn't have any real attackers, our playbooks play their role in a controlled way to produce realistic and varied movement towards the honeypot. Seven (7) playbooks were defined, each with a distinct objective, which together cover the range of tactics of an attacker after initial access.

Playbook	Intent	What it does
recon	reconnaissance	Basic system and user identification
credential_search	credential_harvesting	Search for codes and keys
config_discovery	environment_mapping	Service mapping via configs
privilege_escalation	privilege_escalation	Attempt to increase privileges (sudo, shadow)
data_staging	data_exfiltration	Locating data to export
lateral_movement	lateral_movement	Preparing to switch to other machines
honeypot_fingerprinting	honeypot_detection	Control for sandbox/honeybot

Depth variations: shallow and deep sessions

Each playbook is executed in three depth variations to simulate the fact that in reality some attackers are hasty and superficial, while others are methodical and thorough. The **shallow (shallow)** Camouflage only executes the first, basic commands, like an attacker taking a quick look. The **deep** Camouflage performs the whole sequence, reading the baits as well. The **deeper (deep_plus)** Variation adds additional follow-up commands, like an attacker following the clues it finds. For example, in the credentials lookup playbook, the shallow version simply looks at the personal folder, while the deep version reads .env, database.yml, and .ssh/config.

Slight randomness is added to each run (small differences in the order, number of commands and pauses), so that repetitions of the same playbook do not come out identical. Thus a total of 35 sessions with a real variety of depth was produced. The pronunciation requires at least 15 sessions, so the limit is deliberately exceeded, for more meaningful statistical analysis and clustering.

Why we keep the ground truth separate

The term ground truth means the confirmed, known truth. Because we ourselves define which playbook runs each time, we know for sure what the real goal (intent) of each session was. This known truth is stored in a separate file (`playbook_results/ground_truth.json`).

The important methodological detail is that the honeypot is not aware of this goal. The honeypot only records what happened, i.e. which orders were given, without any intent tag. Thus, in Task 5 the grouping of the sessions is done blindly, only on the behavioral data, and only at the end is it compared to the ground truth to see how well the analysis discovered the real tactics on its own. If, on the other hand, the intent came from the very name of the playbook, the analysis would be tautological, that is, it would confirm something we already knew.

Files implementing Task 4: `03_run_playbooks.py` and the envelope `playbook_results/` (`ground_truth.json`, `playbook_catalogue.json`).

Task 5: Behavior and Intention Analysis

The goal of this stage is to turn raw logs into meaningful knowledge about attackers' behavior. The analysis is done in three steps: first we extract numerical features from each session, then we group the sessions based on them, and finally we single out the deepest, targeted sessions.

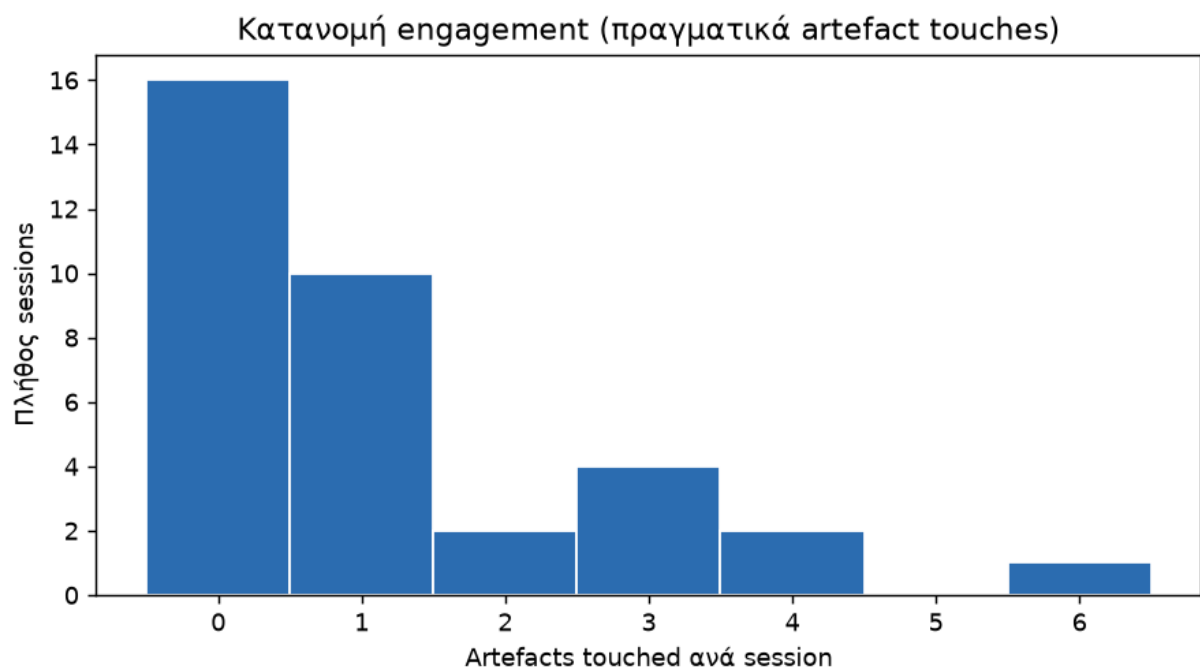
Behavioral features

For each command-in-command session, eight attributes were extracted. Five of them measure the intensity of the engagement: the duration (dwell time), the number of commands, the variety of commands (unique to total), the number of artifacts touched, and the maximum path

depth. The other three measure the type of activity, as ratios: the percentage of read commands, the percentage of system recognition commands, and the percentage of privilege upgrade commands. The combination of intensity and genre allows for the separation of not only how deep, but also what kind of actions each attacker did.

Metric	Average	Description
dwel_time	5.43 s	Session duration
command_count	5.20	Number of commands
cmd_diversity	0.83	Unique/Total Commands
artefacts_touched	1.14	Decoys read
path_depth	2.91	Maximum Stroke Depth

The first and most important finding is that the metric `artefacts_touched` is real. In a minimal static environment with no baits, no read command would return content, so the value would be 0 in all sessions. The presence of the baits caused real engagement, with an average of 1.14 and a maximum of 6 artifacts in a single session. The figure below shows the distribution: several sessions (mostly the shallow ones) did not touch any bait, while a clear subset of deep sessions touched three or more. This dispersion is exactly what makes the analysis interesting.



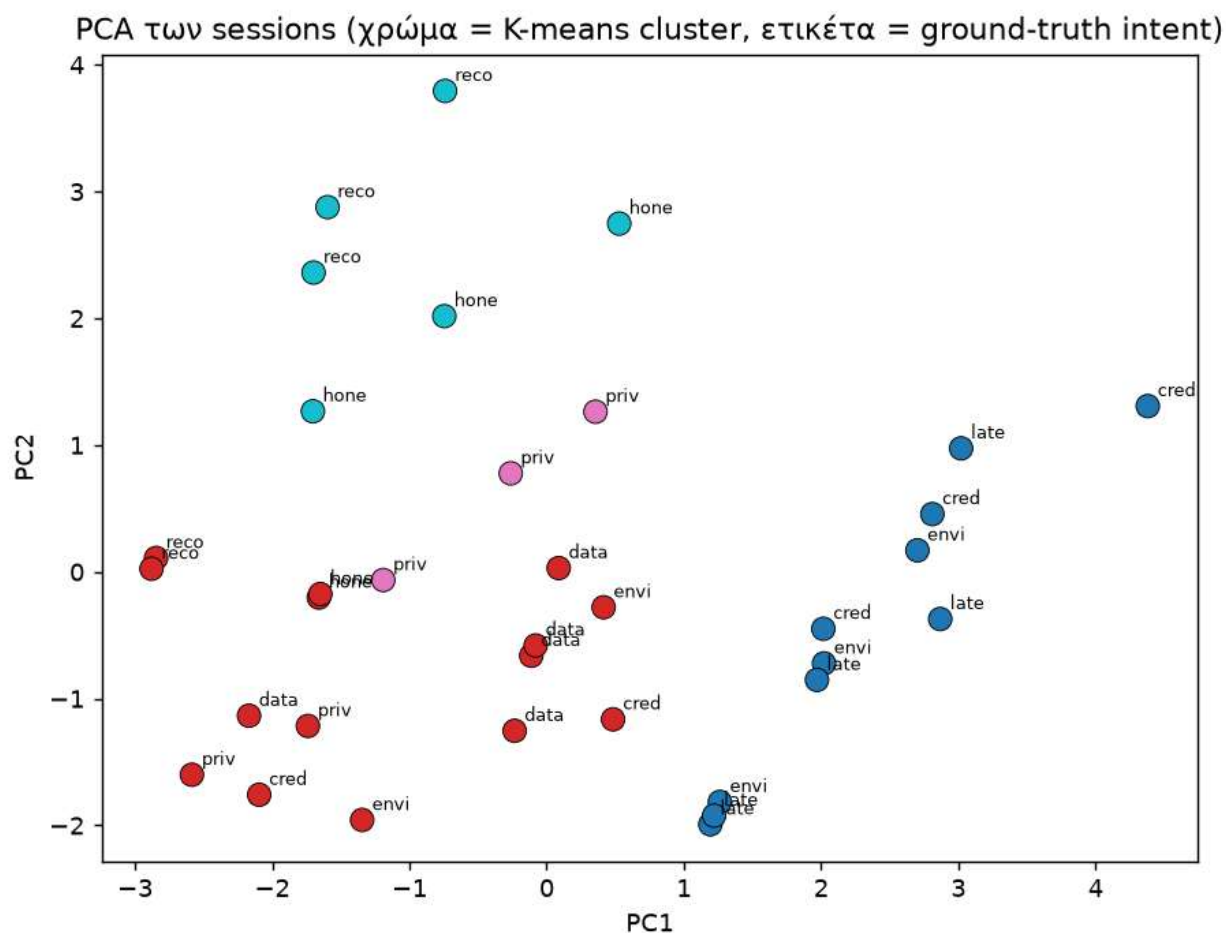
Clustering of sessions

To discover the bias-free behavior patterns, we applied two unsupervised methods on top of the normalized traits: K-means and DBSCAN (lecture 3). K-means asks for the number of teams in advance. We chose four groups (k=4), a value that was tested with the silhouette index and gives the clearest and most interpretable profile distinction. The grouping revealed four clear behavioral profiles.

Team Profile	Size	Average touches	Average orders
Deep readers	11	2.73	5.64
Shallow probes (shallow)	15	0.60	3.67
Deep recognition without reading	6	0.00	7.83
Intermediate probing	3	0.33	6.00

The interpretation is immediately actionable for a security operations center (SOC). Deep readers are the most dangerous group, as they read a lot of baits (an average of 2.73), that is, they are actively looking for sensitive information. Deep recognition executes most commands (7.83) but does not read baits, a pattern typical of recon and fingerprinting, where the attacker examines the system (ps, uname, /proc) without opening any files. Shallow probes are the largest but least risky group, with surface exploration.

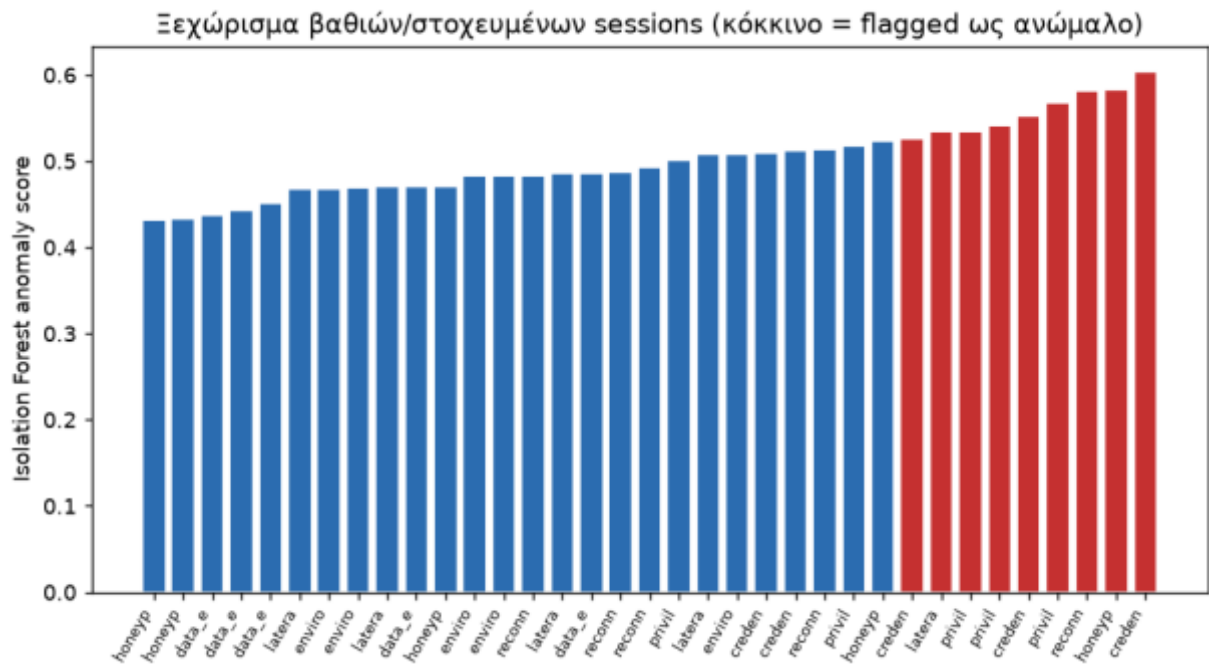
The figure below projects the sessions in two dimensions using the PCA method (lecture 3), which condenses the eight features into two axes for visualization. The color indicates the K-means group and the label the actual intent. It is observed that the sessions spread from left (shallow) to right (deep), and that the groups form distinct areas.



Comparing the teams to the ground truth, the Adjusted Rand Index (ARI) is low (0.185). This indicator measures how much teams coincide with actual intents, with 1.0 meaning perfect agreement. The low price **It's not a failure but a finding**: behavioral traits group sessions by depth and engagement profiles, not by exact semantic intent. Tactics with a similar profile (for example recon and fingerprinting, both of which execute multiple identification commands without reading baits) logically fall into the same group. The question of whether we can predict the exact intent is handled with a supervised model in the optional task, with clearly better results.

Detection of deep sessions (Isolation Forest)

Finally, we used Isolation Forest (lecture 3), an anomaly detection method, to automatically distinguish deep, targeted sessions from shallow, daily sessions. The model scored 9 out of 35 sessions as abnormal. As the **Figure 3**, these correspond to the deepest interactions (credential search, privilege escalation, deep recon), i.e. exactly the ones a security analyst would like to see first. Practically, Isolation Forest works as an automatic priority filter.



Indicative session summaries

Here are three representative summaries, based on actual log data, one from each key profile:

- **Deep Reader (credential_harvesting, deep_plus):** 9 commands in 11.9s, path depth 3, touched 6 baits (.env, database.yml, .ssh/config, known_hosts, bash_history, auth.log). Clear intent to collect credentials and map the network.
- **Deep detection (honeypot_detection, deep):** 8 commands, 0 baits. Run cat /proc/version, ls /.dockerenv, ps aux, ip addr, i.e. background checks to see if it is sandboxed, without opening any files.
- **Shallow probe (reconnaissance, shallow):** 3 commands in 3s (whoami, id, uname). Surface identification, no entanglement with baits.

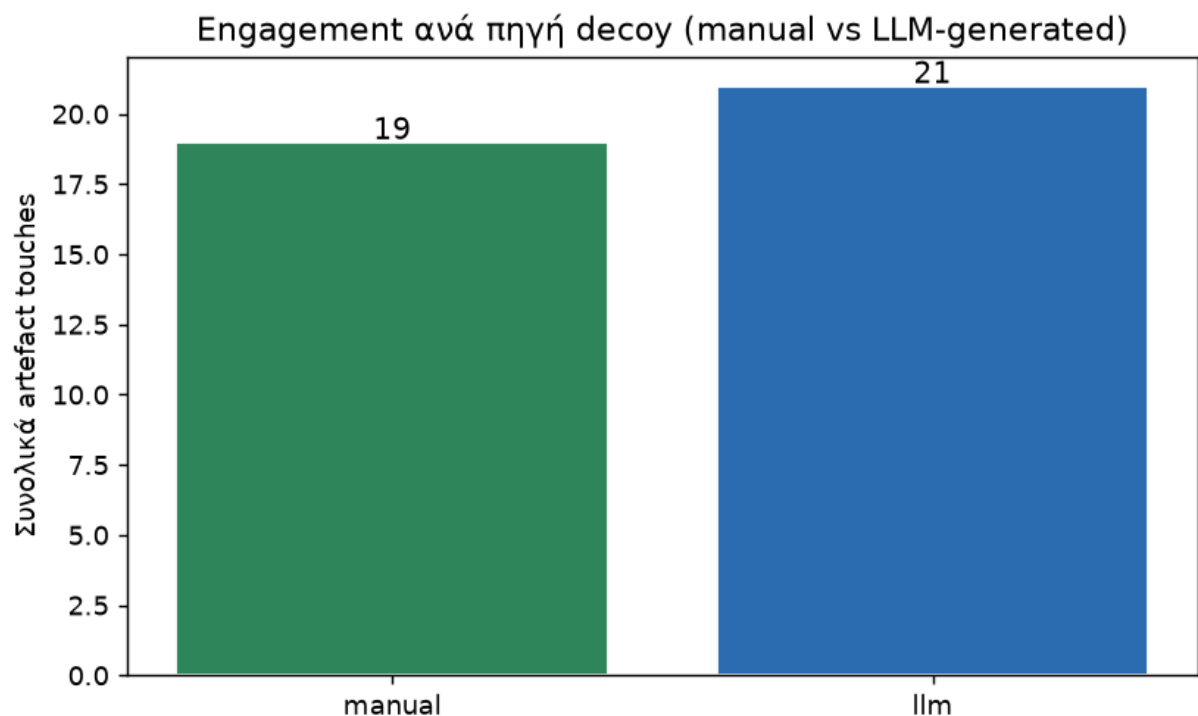
Files implementing Task 5: 04_analyze_sessions.py, the envelope analysis/ (session_features.csv, analysis_report.json, cluster_composition.json) and the file figures/.

Task 6: Deception Quality and Risk of Fingerprinting

This stage evaluates two things: how convincing the baits are (and especially whether LLM-generated ones perform better than manual ones), and how easily an attacker can tell that they are in a honeypot.

Manual vs. LLM-generated baits

To compare the two sources, we measured how much engagement each one got, using the actual artifact touches. Manual baits received 19 touches and LLM-generated 21. The finding is that LLM-generated baits do not lag behind manual baits in engagement, as shown in Figure 4. It needs to be noted, however, that the touches also depend on which paths the playbooks are targeting, not just the attractiveness of the content, so the comparison shows equivalence rather than superiority.



Qualitatively, LLM-generated baits (the runbook, infrastructure notes, ssh config) are richer in narrative and internal consistency, while manual ones are shorter and more standardized. By category, ssh baits (known_hosts, config) received the most engagement (11 taps), followed by credentials (8) and command history (6), which is to be expected, given that these reveal network topology and accesses, i.e. what an attacker is looking for first.

How can honeypot be betrayed (fingerprinting)

For the sake of completeness, the points at which a cautious attacker could perceive to be in a honeypot are listed:

- Shallow file system: the virtual filesystem contains only the baits and a few synthetic files. An extended find/or ls in unpredictable directories reveals the lack of depth.
- Restricted Command Set: Commands outside the supported subset return command not found. An attacker who tries rare commands will detect inconsistencies.
- Uniform response time: responses are almost instantaneous and without realistic latency, which is different from a real system under load.
- No Full Terminal (PTY): tab completion, job control, and full color manipulation/ANSI, which an experienced user would expect, are not supported.
- Static baits and fixed key: content does not change between connections. Periodic regeneration (dynamic refresh) would reduce this mark.

An additional, more general risk of LLM-generated baits is the so-called hallucinations, i.e. the possibility that the model will introduce a detail that contradicts the rest of the environment. In the present work, this risk was mitigated by feeding all prompts with the single system profile and checking the consistency of the generated content.

Files implementing Task 6: 04_analyze_sessions.py (manual analysis vs LLM, analysis_report.json).

Optional Task: Supervised intent classifier

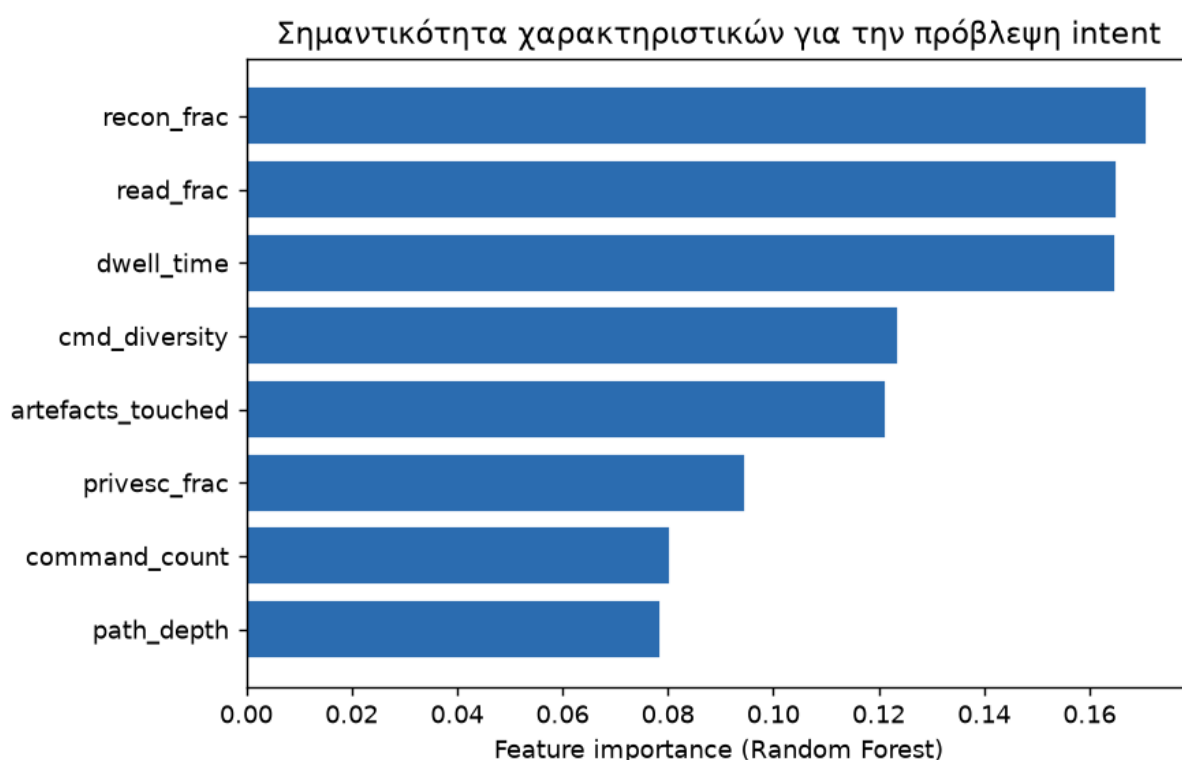
From the optional advanced tasks of the pronunciation, a complete and self-contained one was implemented: a supervised intent classifier. In addition, the comparison of manual vs. LLM-generated baits (Task 6) is also one of the proposed advanced topics, which was covered with real data.

The motivation of the classifier arises directly from Task 5: there, unsupervised clustering grouped the sessions by engagement profiles rather than by precise intent (low ARI). The question

that remains is whether intent is at all predictable by behavior. The answer, with supervised learning, is positive.

A Random Forest model (lecture 3) was used, which learns from examples to match the characteristics of a session to its intent. Because the dataset is small (35 sessions, 7 classes), the evaluation was done using the Leave-One-Out Cross-Validation method: each session is predicted by a model trained on the remaining 34, so that no prediction is made on data that the model has already seen. The model achieved an accuracy of 0.571 and macro-F1 of 0.559, clearly above the random prediction, which for 7 classes would have been around 0.14. That is, the model correctly predicts intent in six out of ten cases.

The figure below shows the importance of the features. The most decisive are the percentage of recognition commands (`recon_frac`), the percentage of read commands (`read_frac`) and the duration (`dwell_time`). This makes intuitive sense: the kind of commands an attacker gives (whether they're looking for a system or reading files) reveals their target. The conclusion is that behavioral traits carry real intent information, which unsupervised clustering could not fully exploit without tags, but the supervised classifier exploits it.



Files that implement the optional task: `05_intent_classifier.py` and the `analysis/intent_classifier.json`.

Task 7: Ethical, Security, and Operational Use

Building a honeypot, and particularly using LLMs to produce persuasively misleading content, raises ethical issues that are addressed based on the pillars of responsible AI (lecture 7).

Isolation and security

The entire system works exclusively in an isolated lab, with the honeypot listening only in the loopback (127.0.0.1). Nothing is exposed on the internet, so the honeypot cannot be used as a pivot point for attacks on third parties, which would turn it from a defensive tool into a danger.

Data Minification

No real credentials, personal data, or live secrets were used. All addresses belong to private zones or documentation zones (RFC 1918 and RFC 5737), and all hosts are fictitious. In logging, no entire credentials are ever stored, only a sample, so that even the logs themselves are not at risk of leakage.

The Ethical Dimension of Using LLMs for Deception

The question of whether the use of LLMs for the production of misleading material changes the ethical profile is essential, and the answer is positive on three points. First, scale: an LLM can produce unlimited, varied, and persuasive content at minimal cost, something impossible manually. Second, persuasiveness: the material is more difficult to distinguish from the real thing, therefore more effective but also more dangerous if it turns to misuse. Third, dual use: the exact same technique that produces baits for defense can produce material for phishing or social engineering. For these reasons, the use was strictly limited to a defensive, isolated context, with fully documented prompts and no production of material that could target real persons or systems.

Business use and transition to production

For true deployment, the system would require four things: enhanced isolation (containerization and strict network exit restrictions), human-in-the-loop oversight in the analysis of findings, clear legal and organizational approval, and periodic regeneration of baits so that they do not become predictable. The risk management cycle (identify, assess, mitigate, monitor) should be continuous, not a one-time action.

Files documenting Task 7: The **present reference**, the **README.txt** (security notes) and comments within the **00_config.py** and **02_ssh_honeypot.py**.

Execution conditions and parameters

- Repeatability: stable seed of randomness (25012) in bait production and playbooks.
- Environment: Python 3.10 and newer (tested on 3.10 and 3.12), with the paramiko, scikit-learn, numpy, pandas, and matplotlib libraries. It works on Linux (Kali) and Windows.
- Network: honeypot listens to 127.0.0.1:2222. All traffic is local.
- Execution: the entire chain runs on a single command (python3 run_all.py), as documented in the README.txt.

Limitations and Threats to Validity

For the correct interpretation of the results, the limits within which the conclusions apply are stated:

- Synthetic attackers: the move is generated by predetermined playbooks, not by real opponents. Real behaviors would be more varied and unpredictable.
- Small sample: 35 sessions is enough to demonstrate, but the classifier's metrics are indicative and would stabilize with more data.
- Limited shell: the simulator covers a realistic but not complete subset of commands, which creates the fingerprinting points of Task 6.

- Static baits: the lack of dynamic regeneration makes the environment predictable in repetitive connections.

Conclusion

A comprehensive and repeatable honeypot analytics framework was constructed, which combines a real SSH honeypot with shell simulation, LLM-enriched baits, structured recording, and behavior analysis based on unsupervised and supervised learning. The central design principle was that the analysis should be based on real facts: because the honeypot actually serves the baits, the metric artefacts_touched is real and not speculative.

Three findings summarize the work. First, LLM-generated baits caused engagement comparable to manual ones (21 vs. 19 touches), making them a viable option for scaling bait production. Second, unsupervised clustering effectively groups sessions by engagement profiles (deep readers, shallow probes, deep identification), something directly usable for prioritization in a SOC, while predicting the exact intent requires a supervised model, which achieved an accuracy of 0.571. Thirdly, capturing fingerprinting points and ethical considerations is integral to a responsible approach. Overall, work shows that amplification with LLMs increases the persuasiveness of a honeypot, but its value ultimately depends on the rigor of the recording and the accuracy of the analysis.

Task Deliverables and Mapping to Files

Task	Implemented by
1. Honeypot design	02_ssh_honeypot.py, 00_config.py
2. Decoy artefacts	01_generate_decoys.py, decoys/, llm_prompts/
3. Session recording	02_ssh_honeypot.py, logs/, SCHEMA.md
4. Attacker playbooks	03_run_playbooks.py, playbook_results/
5. Behavior analysis	04_analyze_sessions.py, analysis/, figures/
6. Realism & fingerprinting	04_analyze_sessions.py, figures/fig5

Task	Implemented by
7. Ethics & Safety	README.txt, comments on 00_config.py /02_ssh_honeypot.py
Optional: intent classifier	05_intent_classifier.py, figures/fig6, fig7

In addition to the code, the submission includes this report, the README.txt, the SCHEMA.md, the dependency list (requirements.txt), the llm_prompts/ folder, and the screenshots/ folder.